Linux Kernel and JVM Runtime Tuning for High-Partition Apache Kafka Clusters

Author : Om Shree | Cloud Engineer | Global Platform Capabilities, FICO

Workload Profile and Scope

This document evaluates Linux kernel and JVM runtime tuning for a high-throughput Apache Kafka deployment with the following operating envelope:

Dimension	Value
Broker instance type	`r5b.16xlarge`
Broker count	12
ZooKeeper node count	6
vCPU per broker	64
Memory per broker	512 GiB
EBS bandwidth per broker	40 Gbps / 5,000 MB/s
EBS IOPS ceiling per broker	~173K IOPS
Network bandwidth per broker	20 Gbps
Topics	~12,000
Partitions	~130,000
Replication factor	3
Total partition replicas	~390,000
Average payload size	~5 KiB
Sustained produce rate	500K+ messages/s
SLA target	200 ms service-level latency
Retention	3 hours to 365 days, topic-dependent
Cleanup policy	Delete and compact, topic-dependent

The deployment is not primarily constrained by raw Kafka capability. It is constrained by multiplicative metadata and replica cardinality. A 500K messages/s workload with 5 KiB payloads implies roughly 2.38 GiB/s of logical producer payload before protocol overhead, compression effects, batching, replication, acknowledgements, TLS, and consumer egress are considered. With replication factor 3, the cluster must absorb approximately 7.15 GiB/s of replicated broker-side disk append pressure before overhead. Even if compression reduces disk bytes, the broker still pays CPU, request scheduling, batching, indexing, replication, and metadata-management costs.

The more serious scaling dimension is partition density. With 130K partitions and RF=3, the cluster owns approximately 390K partition replicas. Distributed evenly across 12 brokers, each broker hosts roughly 32.5K partition replicas and leads roughly 10.8K partitions. That is a large operational surface. Each broker is not merely processing bytes; it is maintaining thousands of log directories, segment files, indexes, replica fetch positions, leader state, ISR state, delayed operations, file handles, memory maps, scheduler tasks, and background maintenance paths.

This document treats the broker as a coupled Linux/JVM/storage/network system. The goal is not to maximize a benchmark number. The goal is to preserve the following invariants under steady load, burst load, rolling restart, broker loss, partition reassignment, replica catch-up, compaction, retention deletion, and cold-cache consumer replay:

1. Produce append latency remains bounded.
2. Fetch latency remains bounded for hot and recently warm data.
3. Replica lag does not amplify into ISR churn.
4. Dirty-page writeback remains smooth rather than cliff-shaped.
5. Page cache remains available for Kafka log traffic.
6. JVM pauses do not violate broker timing assumptions.
7. Kernel queues do not hide overload until the SLA is already breached.
8. Partition cardinality does not exceed mmap, file descriptor, recovery, or controller/ZooKeeper capacity.

This is a tuning document, but it is also a risk document. At this scale, a wrong knob rarely fails alone. It moves pressure into another subsystem.

---

1. Derived Broker Load Model

Before tuning the kernel or JVM, the broker-level load must be made explicit.

1.1 Cluster Arithmetic

```
logical_produce_bytes_per_second
  = 500,000 messages/s × 5 KiB
  ≈ 2.38 GiB/s

replicated_write_bytes_per_second
  = logical_produce_bytes_per_second × replication_factor
  ≈ 7.15 GiB/s

partition_replicas_total
  = 130,000 × 3
  = 390,000

partition_replicas_per_broker
```

```
  = 390,000 / 12
  ≈ 32,500

leader_partitions_per_broker
  = 130,000 / 12
  ≈ 10,833

leader_produce_rate_per_broker
  = 500,000 / 12
  ≈ 41,667 messages/s

leader_payload_per_broker
  = 41,667 × 5 KiB
  ≈ 203 MiB/s

replicated_append_payload_per_broker
  = 203 MiB/s × 3
  ≈ 609 MiB/s
```

These figures are intentionally payload-only. They exclude record batch overhead, Kafka protocol framing, compression ratio variance, index writes, filesystem metadata, replica acknowledgements, producer retries, TLS overhead, consumer fetch traffic, and page-cache misses. The practical implication is that each broker's 5,000 MB/s EBS envelope looks large in the mean case, but recovery and replay can consume it quickly. The 20 Gbps network envelope is more sensitive because the same broker may simultaneously accept producer traffic, send consumer traffic, receive follower replication, send leader replication, and exchange control-plane traffic.

1.2 The Real Scaling Problem: Partition Density

A broker hosting ~32.5K replicas behaves differently from a broker hosting 2K replicas even at the same byte rate. The high-partition broker has:

- more log segments,
- more index and time-index files,
- more file descriptors,
- more mmap areas,
- more replica fetch state,
- more leader/follower state transitions,
- larger metadata propagation cost,
- heavier recovery scanning,
- larger controller/ZooKeeper state surface,
- more background retention and compaction decisions,
- more chances for a small subset of hot partitions to dominate CPU and network.

At this scale, byte throughput is only one axis. Partition cardinality is the more dangerous axis because it creates per-object overhead even when a partition is quiet.

Broker stress = f(bytes/sec, partitions, replicas, segments, retention, compaction, client fanout)

```
Bytes/sec tests disks and NICs.
Partitions test metadata, mmap, FDs, recovery, controller, scheduler, and
heap.
Retention tests storage, segment count, and page-cache working set.
Compaction tests background I/O and CPU isolation.
```

---

2. Kafka Broker Hot Path

Kafka is fast because it is boring in the right places. Producers append to partition logs. Consumers fetch byte ranges. Followers fetch from leaders. The OS page cache absorbs recent writes and serves hot reads. Kafka avoids turning the broker heap into a data cache.

PRODUCE PATH

```
Producer socket
    ↓
Kafka network thread
    ↓
Request queue
    ↓
Kafka request handler
    ↓
Partition append / log segment append
    ↓
Page cache dirty pages
    ↓
Kernel flusher / block layer
    ↓
EBS volume
```

FETCH PATH

```
Consumer or follower fetch
    ↓
Kafka network thread
    ↓
Log read
    ↓
Page cache hit? ── yes ──> socket send / zero-copy path
```

```
          |
          no
          ↓
EBS read → page cache fill → socket send
```

The broker should therefore be tuned as a page-cache and network-transfer machine with a JVM control plane around it. Heap is important, but it is not the data plane. For this workload, the primary design error would be to overfit JVM flags while ignoring page-cache residency, mmap count, file descriptors, dirty-page writeback, and replica recovery.

I. Linux Kernel Tuning for Kafka Brokers

3. Memory Model

Kafka broker memory must be divided into explicit ownership domains.

Domain	Owner	Kafka relevance	Failure mode
JVM heap	JVM	Metadata, request objects, group/transaction state, delayed operations	GC pauses, OOM, request latency spikes
Direct/native memory	JVM + libraries	Network buffers, compression, NIO, TLS, native allocations	Shadow heap, native OOM, container kill
Page cache	Linux kernel	Kafka log segment cache; hot produce/fetch path	Cold reads, follower lag, disk amplification
Kernel memory	Linux kernel	TCP buffers, dentries, inodes, slab, block structures	reclaim stalls, socket pressure, allocation stalls
Mapped indexes	Kafka + kernel VM	Offset/time indexes for log segments	`vm.max_map_count` exhaustion

On `r5b.16xlarge` with 512 GiB RAM, the heap should not be sized as though the broker were an in-memory database. A large heap reduces page cache and can make Kafka slower. For this cluster shape, the correct bias is:

```
Heap:       restrained and stable
Page cache: large and protected
Swap:       disabled
Dirty data: bounded explicitly
Direct mem: bounded and observed
Mmap count: sized from segment cardinality
FD limit:   sized from segment and connection cardinality
```

3.1 Heap vs Page Cache Budget

A defensible starting allocation per broker:

Component	Starting envelope	Notes
JVM heap	24–48 GiB	Start 32 GiB; raise only if metadata/object evidence demands it.
Direct/native memory	8–32 GiB reserved	Depends on TLS, compression, socket buffers, request sizes.
OS + kernel + slab	16–32 GiB reserved	Inodes, dentries, TCP, block layer, page tables.
Page cache	Remainder, usually 380+ GiB	Critical for hot log reads and follower fetches.

A 64–128 GiB heap may look safe on a 512 GiB box, but it is usually a lazy answer. At 32.5K replicas per broker, some heap growth is expected from metadata and delayed operations, but the primary broker data path still wants page cache. Heap must be sized from live-set evidence, not machine size.

Recommended starting JVM memory envelope:

```
-Xms32g
-Xmx32g
-XX:MaxDirectMemorySize=16g
```

Escalation policy:

Condition	Action
Old-gen occupancy stable below 55–60% after full traffic cycle	Keep heap unchanged.
Old-gen steadily grows without traffic correlation	Investigate leak / metadata accumulation before increasing heap.
Young GC pressure high but old-gen stable	Reduce allocation rate or tune young behavior; do not blindly add heap.
Metadata pressure from partitions/groups/transactions is proven	Increase to 48 GiB and remeasure page-cache impact.
Heap above 64 GiB appears necessary	Treat as architectural signal: partition density, client behavior, metadata cardinality, or topic layout is likely excessive.

3.2 Swap

Dedicated Kafka brokers should run without swap. If swap is present for platform reasons, set it so low that it acts as a last-resort safety mechanism rather than a routine memory tier.

```
vm.swappiness = 1
```

A Kafka broker that swaps is not degraded; it is entering an availability failure mode. Swapping request handlers, network threads, or replica fetchers can cause client timeouts, ISR movement, controller instability, and rolling failure under load.

3.3 Dirty Page Writeback

Kafka produces file-backed dirty pages. The kernel later writes them to EBS. If dirty limits are too high, the broker may appear healthy while dirty data accumulates, then experience a writeback cliff. If dirty limits are too low, foreground appenders are throttled too aggressively and throughput suffers.

On 512 GiB memory hosts, ratio-based dirty settings are dangerous because they scale with memory. Use byte limits.

Recommended starting point per broker:

```
vm.dirty_background_bytes = 1073741824      # 1 GiB
vm.dirty_bytes            = 8589934592      # 8 GiB
vm.dirty_expire_centisecs = 3000            # 30s
vm.dirty_writeback_centisecs = 500          # 5s
```

This intentionally begins writeback early and caps dirty exposure. With an average replicated append payload of ~609 MiB/s per broker before overhead, an 8 GiB hard dirty threshold represents only a short writeback buffer. That is the point. The SLA is 200 ms; hiding seconds of dirty data in memory is not free. It becomes future latency.

Trade-off matrix: -

Dirty policy	Benefit	Risk
Low dirty limit	Predictable writeback, less cliff risk	More frequent throttling under bursts
High dirty limit	Higher burst absorption	Larger delayed writeback cliffs
Ratio-based limit	Simple	Unsafe on large-memory brokers
Byte-based limit	Predictable absolute exposure	Must be tuned against device bandwidth

For this cluster, the first production experiment should test 1/8 GiB, 2/16 GiB, and possibly 4/24 GiB dirty envelopes under producer load plus replica recovery. The winning configuration is the one that minimizes p99 append latency and I/O pressure during recovery, not the one with the highest synthetic throughput.

3.4 `vm.max_map_count`

This is a first-class Kafka setting at 130K partitions. Kafka maps log segment index files. Each segment typically has offset-index and time-index mappings. Therefore, mmap pressure scales with segment count, not message rate.

```
partition_replicas_per_broker ≈ 32,500

segments_per_replica = retained_bytes_per_replica / segment.bytes

mapped_index_files_per_broker ≈ 2 × partition_replicas_per_broker ×
segments_per_replica
```

The default Linux value around 65K is structurally unsafe for this partition density. With only one retained segment per replica, a broker is already near:

```
2 × 32,500 = 65,000 mappings
```

That is before active segments, rolled segments, compacted topics, long-retention topics, non-log mappings, shared libraries, heap mappings, thread stacks, and fragmentation of VMA regions. Therefore, `262144` is not a serious ceiling for this workload. It may run until retention distribution or segment cardinality changes, then fail in a way that looks unrelated to throughput.

Recommended:

```
vm.max_map_count = 2097152
```

Minimum acceptable:

```
vm.max_map_count = 1048576
```

Rationale:

Value	Assessment
65,530 / 65,535	Default-class value; unsafe for 32.5K replicas/broker.
262,144	Only acceptable if segment cardinality is tightly controlled.
1,048,576	Reasonable production floor.
2,097,152	Recommended for this workload due to partition density and retention variance.
4,194,304	Acceptable if long retention + small segment sizes are common; monitor VMA overhead.

Validation command:

```
wc -l /proc/$(pidof java)/maps
find /kafka-logs -name '*.index' -o -name '*.timeindex' | wc -l
```

Alerting policy:

Live map usage	Action
>40% of limit	Review segment count trend.
>60% of limit	Capacity risk; inspect retention and segment size.
>75% of limit	Block partition/retention expansion until mitigated.
>85% of limit	Treat as production incident precursor.

3.5 File Descriptors

At this partition count, file descriptors must be sized from file cardinality, not guessed. A partition replica may have multiple segment files, index files, time-index files, transaction indexes, leader-epoch checkpoint files, producer snapshots, and active file handles. Connections also consume descriptors.

Recommended broker service limit:

```
LimitNOFILE=4000000
```

Minimum:

```
LimitNOFILE=1000000
```

Sizing model: -

```
fd_budget ≈ safety_factor × (
    open_segment_files
  + index_files
  + timeindex_files
  + txnindex_files
  + client_connections
  + interbroker_connections
  + log cleaner files
  + JVM/runtime descriptors
)
```

Do not use a low FD ceiling and rely on alerts. FD exhaustion is abrupt. It can surface as log append failures, network accept failures, index access failures, or controller-visible broker instability.

3.6 VFS Cache Pressure and Slab

High partition counts create many dentries and inodes. Kafka benefits from retaining filesystem metadata because log segment operations, retention scans, compaction, and recovery repeatedly touch the same directory and file structures.

Recommended starting point:

```
vm.vfs_cache_pressure = 50
```

This biases the kernel away from aggressively reclaiming inode/dentry cache. Do not set it to pathological values. The target is to reduce metadata churn, not pin every filesystem object forever.

3.7 NUMA and Memory Locality

`r5b.16xlarge` exposes 64 vCPUs and is large enough for NUMA effects to matter. Kafka has network threads, request handlers, replica fetchers, log cleaners, GC threads, and kernel softirq activity. If memory is allocated on one NUMA node and consumed from another, the broker pays latency through remote memory access and cache-line movement.

Policy:

- keep a broker process within a coherent NUMA policy,
- pin or distribute IRQs intentionally,

- avoid concentrating NIC queues on the same CPUs as GC or request handlers,
- use `numactl --hardware`, `numastat`, and `perf c2c` when locality is suspected,
- evaluate `-XX:+UseNUMA` only with measured improvement.

NUMA tuning should not be cargo-culted. A poor pinning policy is worse than no pinning.

3.8 Transparent Huge Pages and Compaction

Kafka brokers should not run with uncontrolled THP behavior for latency-sensitive workloads.

Recommended:

```
echo madvise | sudo tee /sys/kernel/mm/transparent_hugepage/enabled
echo 0 | sudo tee /proc/sys/vm/compaction_proactiveness
```

If explicit JVM huge pages are used, configure them deliberately and test them under production-like load. Automatic compaction can create latency spikes. Kernel compaction is not free; it moves pages and consumes CPU/memory bandwidth.

---

4. `/proc`, PSI, and Host-Level Measurement

At this scale, broker metrics tell what is hurting. Kernel metrics tell why.

4.1 Mandatory Kernel Surfaces

Surface	Use
`/proc/meminfo`	Page cache, dirty memory, writeback, slab, huge pages.
`/proc/vmstat`	Page faults, reclaim, compaction, writeback, pgscan/pgsteal behavior.
`/proc/pressure/cpu`	Runnable delay; detects scheduling pressure invisible in CPU averages.
`/proc/pressure/memory`	Reclaim stalls; leading signal for page-cache/heap conflict.
`/proc/pressure/io`	Block-layer stalls; early warning for append/fetch degradation.
`/proc/<pid>/maps`	VMA count; validates `vm.max_map_count`.
`/proc/<pid>/smaps_rollup`	RSS/PSS split across anonymous and file-backed memory.

Surface	Use
`/proc/interrupts`	IRQ distribution across CPUs.
`/proc/net/snmp`	TCP retransmits, resets, in/out segments.
`/proc/net/netstat`	TCP memory pressure, listen drops, queue overflow symptoms.

4.2 Measurement Contract

Every tuning change must be correlated across four planes:

Plane	Metrics
Kafka request plane	Produce/fetch p50/p95/p99/p999, request queue time, local time, remote time, throttle time.
Replication plane	Under-replicated partitions, ISR shrink/expand rate, follower lag, replica fetch latency.
Kernel plane	PSI, dirty pages, writeback, reclaim, TCP retransmits, disk await, queue depth.
JVM plane	Allocation rate, young pause, mixed pause, full GC, safepoint time, old-gen occupancy.

A tuning change that improves one plane while destabilizing another is not an improvement. For example, raising socket buffers may reduce TCP drops but increase request latency by allowing slow consumers to accumulate larger queues. Raising heap may reduce GC frequency while reducing page-cache hit rate. Raising dirty limits may increase apparent ingest throughput while manufacturing delayed writeback stalls.

---

5. I/O and Storage Path

The R5b family gives high EBS bandwidth relative to standard R5. For `r5b.16xlarge`, the broker has a 40 Gbps / 5,000 MB/s EBS bandwidth envelope and roughly 173K IOPS. That is strong, but the useful question is not whether average write throughput fits. It is whether the broker survives disturbed states.

5.1 EBS Budget

Per broker, the baseline replicated append payload is roughly 609 MiB/s before overhead. On a 5,000 MB/s EBS envelope, this appears comfortable. But the following can overlap:

- producer appends,
- follower replica writes,
- leader reads for follower fetch,
- consumer historical reads,
- log cleaner reads and writes,
- retention deletion,
- partition reassignment,
- broker recovery,
- page-cache misses,
- filesystem metadata writes,
- checkpoint files.

A 200 ms SLA is violated by queueing, not by average throughput. The broker can be under 50% average EBS bandwidth and still violate latency if background work creates bursty queue depth.

5.2 Volume Layout

Do not place all Kafka log traffic onto one large EBS volume unless volume-level queueing and throughput have been explicitly validated. Prefer multiple EBS volumes with Kafka `log.dirs` spread across them, each with provisioned throughput and IOPS sized for recovery.

Example policy: -

Broker resource	Recommendation
EBS type	`io2 Block Express` or sufficiently provisioned `gp3`; choose based on latency and IOPS requirement.
Volume count	6–12 data volumes per broker, validated by queue-depth behavior.
Filesystem	XFS or ext4; benchmark with Kafka segment roll/delete/compaction profile.
Mount options	Avoid unnecessary atime updates; validate `noatime`.
RAID	Only if operationally owned; otherwise prefer Kafka-level distribution across log dirs.

Kafka already distributes partitions across log directories. Use that intentionally. The goal is not to create one giant logical disk; the goal is to prevent one device queue from becoming the broker choke point.

5.3 Segment Size and Retention Coupling

`log.segment.bytes` is not a harmless default. It controls segment roll frequency and therefore affects:

- number of log segments,
- number of mmap index files,
- number of file descriptors,
- retention deletion granularity,
- compaction unit behavior,
- page-cache locality,
- recovery scan surface.

For this workload, a 1 GiB segment is the conservative baseline. Smaller segments improve retention granularity but increase segment cardinality. Larger segments reduce mmap/FD pressure but make retention less precise and can affect compaction/recovery behavior.

Policy: -

Topic class	Retention	Cleanup	Suggested segment policy
Short-window telemetry	3–24h	delete	1–2 GiB segments unless deletion granularity demands smaller.
Business aggregation	days–weeks	delete/compact	1 GiB baseline; adjust by message rate.
Long-retention audit	months–365d	delete/compact	2–4 GiB where acceptable to reduce segment count.
High-churn compacted topics	variable	compact	Avoid tiny segments; tune cleaner capacity.

At 32.5K replicas per broker, segment size is effectively a kernel parameter because it drives VMA and FD cardinality.

5.4 Log Cleaner Isolation

Compaction is not background work in the casual sense. It reads old segments, rewrites retained keys, and competes for disk bandwidth, page cache, CPU, and memory. On this cluster, compaction policy must be tiered by business criticality.

Important parameters:

```
log.cleaner.threads
log.cleaner.io.max.bytes.per.second
log.cleaner.dedupe.buffer.size
log.cleaner.backoff.ms
log.cleaner.min.cleanable.ratio
```

Recommended stance:

- throttle cleaner I/O rather than letting compaction compete freely with produce/fetch,
- isolate compacted topic classes where possible,
- alert on max compaction lag per topic class,
- do not optimize compaction by starving foreground produce/fetch SLA,
- do not set tiny segments on compacted topics without calculating cleaner amplification.

5.5 Recovery Path

The cluster must be sized for broker loss, not normal operation. When a broker restarts or is replaced, it may trigger:

- log recovery,
- leader migration,
- follower catch-up,
- page-cache cold start,
- producer metadata refresh,
- consumer rebalance behavior,
- ZooKeeper/controller pressure,
- replica fetch surges.

Important parameters:

```
num.recovery.threads.per.data.dir
replica.fetchers
replica.fetch.max.bytes
replica.fetch.response.max.bytes
replica.socket.receive.buffer.bytes
replica.socket.timeout.ms
```

Do not set recovery threads high just because CPU exists. Recovery competes with foreground SLA. The correct setting is the highest recovery parallelism that does not violate produce/fetch p99 during broker restart tests.

---

6. Network Path

The cluster's logical producer ingress is ~2.38 GiB/s. Broker network pressure is higher because brokers also replicate data and serve consumers. `r5b.16xlarge` has 20 Gbps network bandwidth. That is not infinite relative to 5 KiB messages and RF=3.

6.1 Broker-Level Network Budget

Per broker average producer leader ingress:

```
~203 MiB/s payload ≈ 1.7 Gbps
```

Per broker replication-related traffic is topology-dependent. Each broker receives data for partitions where it is follower and sends data for partitions where it is leader. With even distribution, replication traffic can add several Gbps before overhead. Consumer egress is workload-dependent and may dominate everything if consumers replay or fan out.

Therefore, the network design must reserve budget for:

- producer ingress,
- follower fetch egress,
- follower fetch ingress,
- consumer fetch egress,
- metadata and controller traffic,
- retries,
- TLS overhead if enabled,
- cross-AZ bandwidth if brokers span AZs.

At 200 ms SLA, bufferbloat is a failure. The target is bounded queues, not maximum buffering.

6.2 Linux Network Sysctls

Recommended starting point:

```
net.core.somaxconn = 65535
net.ipv4.tcp_max_syn_backlog = 65535
net.core.netdev_max_backlog = 250000
net.ipv4.ip_local_port_range = 10000 65000
net.ipv4.tcp_tw_reuse = 1
net.ipv4.tcp_fin_timeout = 15
net.ipv4.tcp_keepalive_time = 300
```

```
net.ipv4.tcp_keepalive_intvl = 30
net.ipv4.tcp_keepalive_probes = 5

net.core.rmem_max = 134217728
net.core.wmem_max = 134217728
net.ipv4.tcp_rmem = 4096 87380 134217728
net.ipv4.tcp_wmem = 4096 65536 134217728
```

These are not final answers. They are a high-ceiling baseline. Kafka-level socket settings still matter.

Kafka broker network parameters to evaluate:

```
num.network.threads=12
num.io.threads=32
queued.max.requests=1000
socket.send.buffer.bytes=1048576
socket.receive.buffer.bytes=1048576
socket.request.max.bytes=<business-dependent>
connections.max.idle.ms=600000
```

Why these numbers:

- default network and I/O thread counts are too conservative for 64-vCPU high-throughput brokers;
- 12 network threads maps better to high connection and fetch load;
- 32 I/O threads gives enough request-handler parallelism without exhausting CPU;
- socket buffers should be raised from small defaults but must be observed for bufferbloat;
- queue depth should absorb bursts but not hide a saturated broker.

Escalation policy:

Symptom	Likely lever
Network processor idle low, request queue rising	Increase `num.network.threads`, inspect NIC IRQ distribution.
Request handler idle low, local time rising	Increase `num.io.threads` or reduce per-request work.
TCP retransmits rising	Inspect ENA, packet drops, cross-AZ path, buffer sizing.
High fetch latency but disk idle	Network/socket/client backpressure likely.
High produce latency with network idle	Request handler, disk, or ISR path likely.

6.3 IRQ, RSS, and Softirq Distribution

On 64 vCPU brokers, NIC queue placement matters. If RX/TX queues or interrupts collapse onto a small CPU subset, Kafka-level network latency appears even while total CPU utilization looks moderate.

Required checks:

```
cat /proc/interrupts
grep . /sys/class/net/eth*/queues/rx-*/rps_cpus
ethtool -l eth0
ethtool -S eth0
mpstat -P ALL 1
sar -n DEV,TCP,ETCP 1
```

The aim is not maximal spreading. The aim is locality: keep packet processing, Kafka network threads, and memory access from fighting across sockets unnecessarily.

II. JVM, GC, and Heap Optimisation

7. JVM Runtime Boundary

Kafka's JVM should be tuned to stay out of the data path. Heap must hold control structures, not the log. For this cluster, heap oversizing is especially harmful because page cache is the broker's real read accelerator.

7.1 Baseline JVM

Recommended starting profile with JDK 17 or JDK 21-class runtime:

```
-Xms32g
-Xmx32g
-XX:+UseG1GC
-XX:MaxGCPauseMillis=100
-XX:InitiatingHeapOccupancyPercent=35
-XX:+ParallelRefProcEnabled
-XX:+AlwaysPreTouch
-XX:MaxDirectMemorySize=16g
-Xlog:gc*,safepoint:file=/var/log/kafka/gc.log:time,uptime,level,tags:filecou
nt=20,filesize=100M
-XX:+HeapDumpOnOutOfMemoryError
-XX:HeapDumpPath=/var/log/kafka/heapdump.hprof
-XX:ErrorFile=/var/log/kafka/hs_err_pid%p.log
```

Notes:

- `AlwaysPreTouch` faults heap pages at startup, reducing runtime page-fault surprises.
- `InitiatingHeapOccupancyPercent=35` begins concurrent marking earlier under G1.
- `MaxGCPauseMillis` is a goal, not a contract.
- `MaxDirectMemorySize` prevents off-heap growth from becoming an invisible second heap.
- GC and safepoint logs must be permanently enabled.

7.2 G1 as Baseline

G1 is the conservative baseline because Kafka should not need enormous heaps if page cache is respected. G1 gives region-based collection, predictable operational history, and good tooling support.

G1 failure modes to watch:

Failure mode	Broker symptom	Response
High allocation rate	Young GC frequency rises	Reduce allocation sources, tune batching/compression paths.
Survivor pressure	Promotion increases	Inspect queued requests and delayed operations.
Humongous allocations	Region fragmentation, pause spikes	Inspect large batches, compression buffers, request max bytes.
Mixed GC bursts	p99/p999 latency degradation	Lower IHOP, increase heap modestly, reduce live set.
Evacuation failure	Severe pause risk	Heap too tight or fragmentation/object profile pathological.
Full GC	Broker-level incident	Treat as Sev-class runtime failure.

7.3 ZGC Consideration

ZGC should be evaluated if:

- heap must exceed 48–64 GiB,
- G1 p999 remains above SLA despite allocation hygiene,
- old-gen/mixed pauses correlate with broker instability,

- CPU headroom exists for concurrent collector work.

Candidate profile on JDK 21:

```
-Xms48g
-Xmx48g
-XX:+UseZGC
-XX:+ZGenerational
-XX:+AlwaysPreTouch
-XX:MaxDirectMemorySize=16g
-Xlog:gc*,safepoint:file=/var/log/kafka/gc.log:time,uptime,level,tags:filecou
nt=20,filesize=100M
```

Trade-off: ZGC reduces stop-the-world sensitivity but consumes concurrent CPU and memory bandwidth. On a broker close to CPU saturation, it may reduce pause time while increasing request queue time. That is not automatically a win.

7.4 Heap Sizing Against Partition Cardinality

At 130K partitions, heap pressure is not just byte throughput. It is metadata surface:

- partition state,
- log objects,
- producer state,
- group coordinator state,
- transaction coordinator state,
- delayed produce/fetch operations,
- ACL/principal metadata,
- quotas,
- metrics cardinality,
- controller metadata propagation,
- client connection state.

Heap expansion must be justified by live-set composition. The correct workflow is:

1. Run representative load.
2. Capture GC log allocation rate and live-set after mixed cycles.
3. Capture heap histogram during steady state and under rebalance/recovery.
4. Confirm page-cache hit impact before increasing heap.
5. Increase heap in 8–16 GiB increments only when live-set evidence demands it.

Heap sizing table:

Heap	Assessment
16 GiB	Likely too small for this partition density.

Heap	Assessment
32 GiB	Recommended starting point.
48 GiB	Reasonable if metadata/live set justifies it.
64 GiB	Upper conservative G1 range; test page-cache cost.
>64 GiB	Prefer ZGC evaluation or partition-density redesign.

---

8. Eden, Young Gen, Old Gen, and Kafka Pauses

Kafka allocation is dominated by request/response processing, protocol decoding, record batch handling, compression/decompression, authorization, metrics, and coordination state. The ideal Kafka object dies young. When it does not, it usually means the broker is queueing work.

```
Request arrives
    ↓
Temporary protocol / buffer / batch objects
    ↓
Eden
    ├── dies quickly: good
    ├── survives due to queueing: survivor pressure
    └── promoted due to retention: old-gen pressure
```

Pause analysis must be mapped to broker symptoms:

JVM event	Kafka symptom
Frequent young GC	Allocation-rate problem; short spikes in request latency.
Rising survivor occupancy	Queues retaining request objects too long.
Rising old-gen occupancy	Metadata growth, delayed operations, leak, or excessive retention of request state.
Mixed GC bursts	p99/p999 produce/fetch degradation.
Full GC	Broker should be considered unhealthy.
Safepoint non-GC pause	Latency spike without heap explanation.
Concurrent GC CPU contention	Request queue time rises despite low STW pause.

For a 200 ms SLA, the practical budget is far below 200 ms for any single internal component. GC should not consume the full SLA. A serious target is:

Runtime event	Target
Young GC p99	<25 ms
Mixed GC p99	<50 ms
Safepoint p99	<10 ms
Full GC	0 occurrences
GC CPU overhead	Stable and explainable

These are operational targets, not JVM promises. If the broker cannot hold them under recovery, reassignment, and compaction, the runtime is not tuned.

---

9. TLB, Huge Pages, and Choke-Point Isolation

TLB tuning must be treated as a diagnostic method, not a superstition. Kafka has multiple memory domains with different translation behavior:

Domain	Backing	Access pattern	TLB relevance
JVM heap	anonymous memory	object traversal, allocation, GC scanning	high during GC and allocation bursts
Direct buffers	native/anonymous	network/compression paths	high under produce/fetch load
Page cache	file-backed	log segment reads/writes	high under hot fetch/cold replay
Mmap indexes	file mappings	offset/time lookup	high with high segment cardinality
Kernel slabs	kernel memory	TCP, VFS, inode/dentry	indirect but important

The TLB question is: which domain is paying translation cost?

9.1 Measurement Procedure

Run separate profiles for:

1. steady produce,
2. hot consumer fetch,
3. follower replication catch-up,

4. cold historical replay,
5. compaction-heavy interval,
6. broker restart and log recovery,
7. partition reassignment.

Collect:

```
perf stat -e
cycles,instructions,stalled-cycles-frontend,stalled-cycles-backend \
  -e dTLB-loads,dTLB-load-misses,iTLB-loads,iTLB-load-misses \
  -e page-faults,minor-faults,major-faults \
  -p $(pidof java) -- sleep 60
```

Interpretation:

Signal	Likely interpretation
DTLB misses spike during GC	Heap traversal locality problem.
DTLB misses spike during hot fetch	Page cache / socket path, not heap.
DTLB misses spike during compression	Direct/native buffer path.
Major faults rise	Cold page-cache reads or memory pressure.
Backend stalls rise with low GC	Memory subsystem / I/O interaction.
IPC falls during compaction	Cleaner I/O and cache interference.

9.2 Explicit Huge Pages

Candidate only after evidence:

```
-XX:+UseLargePages
-XX:LargePageSizeInBytes=2m
```

Operational cautions:

- reserve huge pages at boot,
- use `AlwaysPreTouch`,
- do not rely on runtime compaction to manufacture huge pages,
- validate NUMA placement,
- compare GC phase time and Kafka p99/p999, not only TLB counters.

Huge pages are a good result only if they improve broker latency or CPU efficiency without reducing operational flexibility. If they improve GC scan locality but page-cache misses dominate fetch latency, the tuning is aimed at the wrong layer.

III. Kafka Broker Configuration for This Cluster Shape

10. Baseline Broker Parameters

The following is a starting profile, not a final config. It assumes dedicated brokers, 64 vCPU, 512 GiB memory, high EBS bandwidth, large partition cardinality, RF=3, and mixed retention.

```
# Threading
num.network.threads=12
num.io.threads=32
num.recovery.threads.per.data.dir=2
background.threads=16
queued.max.requests=1000

# Sockets
socket.send.buffer.bytes=1048576
socket.receive.buffer.bytes=1048576
socket.request.max.bytes=104857600
replica.socket.receive.buffer.bytes=1048576
replica.socket.timeout.ms=30000

# Replica fetch
num.replica.fetchers=8
replica.fetch.min.bytes=1
replica.fetch.wait.max.ms=500
replica.fetch.max.bytes=10485760
replica.fetch.response.max.bytes=104857600

# Log segment baseline
log.segment.bytes=1073741824
log.retention.check.interval.ms=300000
log.index.size.max.bytes=10485760
log.index.interval.bytes=4096

# Durability / ISR
min.insync.replicas=2
unclean.leader.election.enable=false
replica.lag.time.max.ms=30000

# Cleaner baseline, must be class-tuned
log.cleaner.threads=8
log.cleaner.io.max.bytes.per.second=<set-per-broker-after-benchmark>
log.cleaner.dedupe.buffer.size=<memory-budgeted>
```

```
log.cleaner.min.cleanable.ratio=0.5

# Controlled shutdown / recovery
controlled.shutdown.enable=true
```

10.1 Threading Trade-offs

`num.network.threads` and `num.io.threads` should not be tuned by CPU count alone. They should be tuned from idle ratio and queueing behavior.

Parameter	Too low	Too high
`num.network.threads`	Socket read/write backlog, network processor saturation	CPU contention, context switches
`num.io.threads`	Request queue buildup, local processing latency	Lock contention, cache pressure
`num.replica.fetchers`	Follower lag, slow recovery	Network/disk contention
`num.recovery.threads.per.data.dir`	Slow restart/recovery	Foreground SLA damage during recovery
`background.threads`	Retention/deletion delays	Background CPU interference

The correct target is not maximum parallelism. It is enough parallelism to avoid queues under normal load, plus controlled degradation under recovery.

10.2 Partition Density Warning

130K partitions is a major design decision. With RF=3 and 12 brokers, ~32.5K replicas per broker is high. The cluster can be made to run, but the operational margin is thinner than the hardware suggests.

Risks:

- broker restart takes longer,
- controller/ZooKeeper metadata pressure rises,
- leader election events are heavier,
- partition reassignment is dangerous without throttling,
- mmap and FD pressure become first-class limits,
- compaction and retention scans grow,
- metrics cardinality becomes expensive,
- small hot-partition skews are harder to isolate.

Staff-level conclusion: if 130K partitions are business-required, tune the host aggressively. If they are an accidental consequence of topic-per-tenant or topic-per-entity design, fix the data model before pretending sysctl can save the system.

---

11. ZooKeeper and Control Plane Risk

A 6-node ZooKeeper ensemble is unusual because ZooKeeper quorum systems generally favor odd node counts. Six nodes require four for quorum, while five nodes require three. The sixth node increases quorum cost without improving failure tolerance in the way people often assume.

For a 130K-partition Kafka cluster, the control plane is a material risk. Watch:

- controller event queue time,
- ZooKeeper request latency,
- ZooKeeper outstanding requests,
- watch count,
- session expirations,
- leader election duration,
- ISR change rate,
- metadata propagation delay.

If the deployment is on a Kafka version that supports KRaft maturely for the organization's requirements, migration should be considered separately. This document assumes ZooKeeper remains present because the supplied architecture includes it.

IV. Recommended Host Profile

12. `/etc/sysctl.d/99-kafka.conf`

```
# VM / memory
vm.swappiness = 1
vm.dirty_background_bytes = 1073741824
vm.dirty_bytes = 8589934592
vm.dirty_expire_centisecs = 3000
vm.dirty_writeback_centisecs = 500
vm.max_map_count = 2097152
vm.vfs_cache_pressure = 50
vm.compaction_proactiveness = 0

# Network backlog and TCP
net.core.somaxconn = 65535
net.ipv4.tcp_max_syn_backlog = 65535
```

```
net.core.netdev_max_backlog = 250000
net.ipv4.ip_local_port_range = 10000 65000
net.ipv4.tcp_tw_reuse = 1
net.ipv4.tcp_fin_timeout = 15
net.ipv4.tcp_keepalive_time = 300
net.ipv4.tcp_keepalive_intvl = 30
net.ipv4.tcp_keepalive_probes = 5

# Socket memory ceilings
net.core.rmem_max = 134217728
net.core.wmem_max = 134217728
net.ipv4.tcp_rmem = 4096 87380 134217728
net.ipv4.tcp_wmem = 4096 65536 134217728
```

13. systemd Limits

```
[Service]
LimitNOFILE=4000000
LimitNPROC=524288
TasksMax=infinity
```

14. THP Policy

```
echo madvise > /sys/kernel/mm/transparent_hugepage/enabled
echo never > /sys/kernel/mm/transparent_hugepage/defrag
```

If explicit huge pages are adopted, reserve them intentionally and document the heap-to-hugepage mapping. Do not mix uncontrolled THP defrag with latency-sensitive Kafka brokers.

V. Validation Plan

15. Load Scenarios

The configuration is not valid until it passes disturbed-state tests.

Scenario	Required observation
Steady 500K+ TPS produce	Produce p99 within SLA; no dirty-page climb.
Hot consumer fetch	Fetch p99 within SLA; no network queue saturation.
Cold historical replay	Disk pressure bounded; hot producers not starved.
One broker restart	ISR stable; recovery bounded; no cluster-wide p99 collapse.

Scenario	Required observation
Partition reassignment	Throttle effective; foreground SLA preserved.
Compaction surge	Cleaner lag controlled; produce/fetch latency bounded.
Retention deletion storm	Metadata and I/O spikes contained.
ZooKeeper disruption	Session stability and controller behavior acceptable.

16. Hard Gates

The deployment should not be considered production-ready if any of the following occur during validation:

Gate	Failure criterion
Full GC	Any occurrence under normal or recovery load.
Major swap activity	Any sustained activity.
mmap usage	>75% of `vm.max_map_count`.
FD usage	>70% of `LimitNOFILE`.
Dirty pages	Repeated climb to `dirty_bytes`.
PSI I/O	Sustained non-zero pressure correlated with p99 breach.
TCP retransmits	Persistent increase under steady load.
ISR churn	Repeated ISR shrink/expand during normal load.
ZooKeeper latency	Sustained request latency that correlates with broker metadata events.

17. Operator Commands

```
# mmap pressure
wc -l /proc/$(pidof java)/maps
find /kafka-logs -name '*.index' -o -name '*.timeindex' | wc -l

# file descriptors
ls /proc/$(pidof java)/fd | wc -l

# page cache / dirty writeback
grep -E 'Cached|Dirty|Writeback|Slab|SReclaimable|SUnreclaim' /proc/meminfo
```

```
cat /proc/vmstat | egrep
'pgscan|pgsteal|dirty|writeback|allocstall|compact|pswp'

# pressure stall information
cat /proc/pressure/cpu
cat /proc/pressure/memory
cat /proc/pressure/io

# network
sar -n DEV,TCP,ETCP 1
cat /proc/net/snmp
cat /proc/net/netstat

# disk
iostat -x 1

# JVM
jcmd $(pidof java) GC.heap_info
jcmd $(pidof java) VM.native_memory summary
jcmd $(pidof java) Thread.print
```

Conclusion

For this cluster, the core engineering problem is not whether Kafka can process 500K messages per second. The core problem is whether 12 brokers can carry 390K partition replicas, mixed retention up to 365 days, compaction and deletion, recovery traffic, page-cache churn, mmap pressure, file descriptor pressure, JVM allocation pressure, and ZooKeeper metadata pressure while preserving a 200 ms SLA.

The correct tuning posture is aggressive but bounded:

- keep heap restrained and evidence-sized,
- preserve page cache as a first-class Kafka data-plane component,
- set `vm.max_map_count` for segment cardinality, not defaults,
- set file descriptors for partition and connection cardinality,
- use byte-based dirty limits,
- isolate compaction and recovery from foreground SLA,
- tune network queues without manufacturing bufferbloat,
- measure PSI and hardware counters, not only Kafka dashboards,
- treat full GC, swap, mmap exhaustion, FD exhaustion, dirty-page cliffs, ISR churn, and ZooKeeper latency as design failures, not operational surprises.

The highest-level trade-off is simple: hardware buys headroom, but partition cardinality spends it continuously. At 130K partitions, the broker is not just a log append service. It

is a metadata-heavy, file-heavy, mmap-heavy, queue-sensitive distributed systems node. The tuning must reflect that reality.